

Verifiable Secret Sharing

2025

1 Introduction

So far we have looked at solutions for sharing a secret as well as as computing on secret inputs whilst assuming that an adversary is “semi-honest”. This is a term that we defined before, and it means that an adversary may only passively take over parties. In particular, if they corrupt a party (or multiple parties), then they see all the actions of these parties and their internal randomness (should this be required, like in the case of a degree reduction after a multiplication), but they cannot interfere with the contributions of those parties. Thus, they cannot change values that these parties produce/contribute to the overall protocol.

We now look at the idea of a technique by which we can deal with active adversaries (we called them malicious), in order to come up with a secret sharing protocol that enables honest parties to check their shares, and if they notice a problem to abort the protocol. This is called **verifiable secret sharing** (VSS).

As an example of a deployed VSS we will look at Feldman’s VSS, which is based on Shamir’s secret sharing and it uses the discrete logarithm problem as part of including something that is verifiable. Hence we make a brief detour, and recap the discrete logarithm problem.

1.1 Discrete logarithm

Discrete Logarithm Problem. Let $(G, \times)$ be an Abelian group. Given $g, h \in \mathbb{G}$, find an x (if it exists) such that

$$g^x = h.$$

The hardness of this problem depends on the group $\mathbb{G}$.

Let us recap the concept of “hardness”. In computer science, we typically relate “hardness” to the amount of computational resources that are necessary for an algorithm to do its job. Thus when we say that the discrete-logarithm problem is hard, then we mean that our best algorithms for finding a solution take time/memory that are beyond what is practically feasible (in the lifetime of our universe).

Many “hard problems” that we use in cryptography are of this nature: we understand

our best solving algorithms, which means that we know they have exponential (in the input size) runtime and/or memory requirements. Therefore, we can configure our choice of group.

For instance, we know that a very easy instance for the discrete-logarithm problem is when we choose the group of integers modulo n with the addition operation $(\mathbb{Z}_n, +)$. So this choice is not suitable for cryptography. A good choice for cryptography is the group of numbers modulo a prime with the multiplication as group operation $(\mathbb{Z}_p, \cdot)$.

NP hard problems. Another common concept in computer science is that of complexity classes, one of which also has the word “hard” associated with it. In this context, a hard problem is one for which we know that the solution is in the class of computationally hard problems (aka NP-hard problems). Such problems are very suitable for cryptography in theory: however cryptography is ultimately a practice driven subject and a cryptosystem that has superior security but leads to inefficient (= not deployable) implementations is useless.

2 Verifiable Secret Sharing

Let us recap on the two practical use cases of secret sharing that we considered so far. We defined secret sharing such that there is somewhere, somehow a secret value, which is shared out so that the parties holding the shares have never seen the actual secret value. To solve this in a straightforward way¹, there would need to be another party involved, the so-called “dealer” who has the secret, and who generates the shares. All parties would need to trust this dealer, and clearly if the dealer is compromised, then there is nothing that we can do to protect the secret. In a real life situation, this dealer would most likely be a hardware security module, in which secrets are internally generated as well as the shares².

Thus when considering malicious adversaries we are now going to consider that share-holding parties could be corrupted and that the dealer could be corrupt. Honest parties will behave exactly according to the protocol, whilst corrupted parties could send wrong values/information.

Let us consider how to deal with this situation in a secret sharing scheme. We allow $t - 1$ parties to be corrupt (this implies that they can also collude) and the dealer. We want the following properties to hold:

¹It is possible that a shared secret is implicitly defined via generating shares. But in a case where the secret is used to control access to something, then the secret must at some point be known and placed into the something.

²Notice that this is quite different to when we use secret sharing for a multiparty computation, where each party knows and shares their inputs. In this case there is no such dealer required.

1. **Reconstruct possible:** t uncorrupted users can successfully recover the secret s ,
2. **Correct secret:** if the dealer was honest, then the recovered secret is equal to the secret of the dealer,
3. **Share privacy:** if the dealer was honest, then $t - 1$ corrupted parties learn no additional information about the secret.

There are several solutions to this problem available, and a rather popular one goes back to a paper by Paul Feldman that was published in 1987. He explains his idea, and how it can be realised using different “hard problems”. One such hard problem is the discrete logarithm problem, and we will base our description on this version.

3 Feldman’s scheme

We choose a field in which the discrete-logarithm problem is hard, e.g. $(\mathbb{Z}_p, +, \cdot)$, and publish the prime p , and a generator of the field g . In practice this means that the dealer and parties must make this choice. They also need to agree on a function $H(x)$, from which we hope that it behaves akin to a random function. By this we mean that it is a function that upon input x outputs a value from its image space more or less at random. There are no (efficiently computable) functions that really have this property, but cryptographic hash functions are a widely accepted real world choice. Finally, we also assume that when the dealer issues a “broadcast message” then all parties will receive this message.

We want to tolerate at most $t - 1$ corrupted parties, and we want that t honest parties can reconstruct. This means that we need at least $(t - 1) + t$ shares when we set up a scheme.

The share function is now an interactive protocol. Reconstruct requires, as in the Shamir scheme, t honest users to collaborate.

3.1 Protocol VSS.Share

1. Share distribution. To share a secret message m the dealer chooses uniform coefficients $a_0, a_1, \dots, a_{t-1} \in \mathbb{Z}_p$. These coefficients define the secret polynomial $f(x) = \sum_{j=0}^{t-1} a_j x^j$. Share i is the i -th point on the polynomial: $s_i = f(i) = \sum_j a_j i^j$. Party P_i gets their respective shares via a private channel between the dealer and the party³.

Furthermore the dealer broadcasts the values $A_0 = g^{a_0}, \dots, A_{t-1} = g^{a_{t-1}}$, and the value $c = H(a_0) \oplus m$.

³To keep our notation uncluttered we assume here now that each party gets just one share. So s_i is the share of party P_i .

2. Share verification. Each party now verifies that their share satisfies

$$g^{s_i} \equiv \prod_{j=0}^{t-1} (A_j)^{i^j} \pmod{q}.$$

1. If party P_i finds that their share does not satisfy the equation then they broadcast a complaint.

Abort: If more than $t-1$ parties complain, then honest parties abort the protocol.

2. All parties P_i who complained receive their s_i again via a public broadcast.

Abort: If the dealer refuses to respond.

Abort: If any broadcast value s_i does not satisfy the verification equation.

3.2 Protocol VSS.Reconstruct

Reconstruction will only be carried out if the dealer has not been disqualified previously (i.e. the share protocol successfully completed).

1. Share verification. A group of parties come together to reveal the secret. In this group there need to be at least t honest parties (i.e. parties who did not complain). The share of each user gets verified according to the share verification equation⁴. If at least t shares can be verified, then they proceed to reconstruct, otherwise they abort.

2. Reconstruct. Reconstruct works according to **Stn.Reconstruct** to derive a_0 . Using a_0 c the t honest users can reveal via $m = H(a_0) \oplus c$.

3.3 Correctness

An honest dealer is never disqualified (see the security argument made subsequently). Assuming that the dealer has not been disqualified, then the value that can be recovered by t honest parties who hold verified shares is equal to m . We set up the scheme such that t such shares s_i are t points on a polynomial. In particular given the construction, we know that they are points on a polynomial $f(x)$ and that there is only one such polynomial $f(x)$. Therefore the t parties will reconstruct the polynomial $f(x)$ that was defined by the dealer in the share generation. This also implies that they can find a_0 and with this, and the public knowledge of c and $H()$ they will find m .

⁴We must verify shares because it could be that a corrupted party did not complain during the Share protocol

3.4 Security argument

Because this protocol relies on the hardness of the discrete-logarithm problem over $\mathbb{Z}_q$, we can no longer guarantee perfect share secrecy. Our guarantee for the secrecy of shares is only computational in this protocol.

Notice that if the dealer is honest then:

$$\prod_{j=0}^{t-1} (A_j)^{i^j} = \prod_{j=0}^{t-1} (g^{a_j})^{i^j} = g^{\sum_{j=0}^{t-1} a_j \cdot i^j} = g^{f(i)} = g^{s_i} \pmod{q}.$$

Therefore all parties can in fact compute g^{s_i} for all values of i . Party P_i can check that their s_i satisfies the verification equation, but they cannot check that any other shares from parties P_j satisfy the respective equation.

If the dealer is honest then all parties got correct shares then complaints indicate that the parties who complain are corrupt. Thus at most $t - 1$ complaints should emerge, and these parties In this case the best they can do is to abort the protocol, which is what happens in the share verification step.

If more than $t - 1$ users complain, then the dealer must have been corrupted. This is because we assume that there are at most $t - 1$ corrupted parties (and they could complain even though they received a correct share), and so if more than $t - 1$ complain, then some honest parties must have received an incorrect share which can only indicate that the dealer has been corrupted. This situation is identified and dealt with in the share verification step of the protocol.

Putting these observations together we can now argue about the privacy of shares.

We know that $t - 1$ corrupted parties cannot reconstruct the secret polynomial. However g^{a_0} is known to all parties. Any of the corrupted parties can therefore try and solve the discrete-logarithm problem over $\mathbb{Z}_q$. Here we can see that our security argument can no longer be about perfect secrecy of shares: we can only give computational security. Assuming that the discrete-logarithm problem is hard, even $t - 1$ corrupt parties cannot extract a_0 . Therefore they only have g_0^a , and they have c . But c is close to a one-time pad encryption of m because $H(a_0)$ looks like a random value. Therefore the corrupted parties do not learn any additional information about m by seeing c .